CS 210

Name: Elijah Griffis RedID: 1 3 2 9 2 5 7 5 1

Program 2 Analysis

My Github repository: https://github.com/eliG45/Program2

1. Briefly define a *queue* data structure.
   A queue is a first in, first out data structure, so items are added to the end and removed at the front

   What evidence did you find that a *queue* container was operating in the load balancer?
   The queue uses queue<request> and front(). The PIDs of the first several requests were roughly from 1 to 10 which means they were in order like a queue. Older requests that finished got assigned new ones so the PIDs are not exactly in numerical order. This is the behavior of a queue's first in first out behaviour.

2. Briefly define a *stack* data structure.
   A stack is a last in, first out data structure, so items are both added to the front and removed at the front

   What evidence did you find that a *stack* container was operating in the load balancer?
   The stack uses stack<request> and top() before pop(). The first several PIDs were instead from the end rather than the front, like 10, 9, 8, etc.. Similar to the queue, the order is not perfect since some requests finish faster and are assigned at a different time. This still shows the last in first out behaviour of a stack.

3. Briefly define a *priority queue* data structure.
   Priority queues store items based on their priority rules instead of arrival order, where the highest priority item is selected first.

   What evidence did you find that a *Max priority queue based on process time* was operating in the load balancer?
   It uses priority_queue<request, vector<request>, MaxProcessTime> and requests that completed earlier had very high process times. This means that lower process time requests had to wait instead which follows the MaxProcessTime rule.

   What evidence did you find that a *Min priority queue based on process time* was operating in the load balancer?
   It uses priority_queue<request, vector<request>, MinProcessTime> and requests that completed early had little to no process times. Requests with shorter run times occurred more frequently. This follows the behavior of the MinProcessTime rule.

   What evidence did you find that a *Max priority queue based on process ID* was operating in the load balancer?
   It uses priority_queue<request, vector<request>, MaxPID> and requests are made based on the highest PID of the requests. However there is still not a perfect order since it is based on completion time of each request so some complete faster than others. This means lower PIDs remain in queue for longer.

Evidence can be a description of the output specific to the data structure and/or an annotated image.